



Blackout Math

Turjo and Urjo are two exceptionally good brothers. One lazy Friday afternoon, the power went out, leaving them completely bored with nothing to do. To pass the time, Urjo decided to invent a math game to challenge Turjo's calculating speed.

Urjo grabs a piece of paper and writes down two starting numbers, A and B . He then tells Turjo that he is generating a special sequence. Since standard addition is far too easy for them, Urjo uses the bitwise XOR operator ($\oplus$) to generate the next terms.

Let F be a sequence. The sequence is defined by Urjo as follows:

$$F_1 = A$$

$$F_2 = B$$

$$F_i = F_{i-1} \oplus F_{i-2} \quad \text{for all } i \geq 3$$

The rules of the game are simple: Urjo will suddenly shout a number N .

To win the game and prove that he is the superior brother, Turjo must instantly shout back the exact value of the N -th term of the sequence, denoted by F_N .

He needs your help to write a lightning-fast program so he can beat his brother and win the game!

Input Format

Input is given in the following format:

- line 1: $N \ A \ B$

Output Format

- line 1: a single integer denoting the exact value of F_N .

Constraints

- $1 \leq N \leq 10^{18}$
- $0 \leq A, B \leq 10^{18}$

Subtasks

Subtask	Score	Additional constraints
1	5	$N \leq 3$
2	7	$A = 1$ and $B = 1$ and $N \leq 10$
3	8	$N \leq 20$
4	10	$N \leq 1000$
5	30	$N \leq 10^5$
6	40	No additional constraints

Examples

Example 1

Input

3 6 7

Output

1

Example 2

Input

5 9 11

Output

11

Explanation

Example 1: $N = 3, A = 6, B = 7$

- $F_1 = 6$
- $F_2 = 7$
- $F_3 = F_2 \oplus F_1 = 7 \oplus 6 = 1$

Example 2: $N = 5, A = 9, B = 11$

- $F_1 = 9$
- $F_2 = 11$
- $F_3 = F_2 \oplus F_1 = 11 \oplus 9 = 2$
- $F_4 = F_3 \oplus F_2 = 2 \oplus 11 = 9$
- $F_5 = F_4 \oplus F_3 = 9 \oplus 2 = 11$

Note

Bitwise XOR($\oplus$) compares two numbers in their binary representation. It evaluates each pair of bits according to the following truth table:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

For example, $6 \oplus 7 = 1$:

- 6 in binary is 00110
- 7 in binary is 00111

If we apply the **bitwise XOR** operator ($\oplus$), it yields 00001, which is 1 in decimal.